

衛星データ解析技術研究会 技術セミナー（応用編）

第5回 高さデータ①

リモートセンシングと高さデータのフォーマット・ 変換処理

2026年7月24日（金）

到達目標：リモートセンシングを起点として高さデータが生成される流れを理解する／GDALでWebGISに利用できる形式（COG・Terrain RGBタイル）へ変換できる

本日の流れ（2.5h）

時間	内容
00:00–00:15	前回の振り返りと本日の位置づけ
00:15–00:50	Part 1：高さデータはどう作られるか（InSAR / LiDAR / SfM）
00:50–01:20	Part 2：フォーマットの構造（GeoTIFF / COG / LAS / LAZ）
01:20–01:30	休憩
01:30–01:45	Part 3：入手できる高さデータソース
01:45–02:20	Part 4：ハンズオン GDAL変換パイプライン → Terrain RGBタイル
02:20–02:30	まとめ・第6回予告

前回（第4回）の振り返り

- 3Dデータのフォーマット整理：3D Tiles / glTF / Potree / COPC
- CloudCompare → PotreeConverter → deck.gl のパイプライン
- 3DGSの現状（Scaniverse / Luma AI、CesiumJS統合の動向）

第4回で扱った点群は「点の集まりのまま」扱う世界だった

第5回・第6回は、それをグリッドに整理した **高さデータ（DEM/DSM）** を扱う

- 点群をラスター化（グリッド化）したものがDSM/DTMの代表的な作り方のひとつ
- 第4回のLiDAR点群処理と地続きの内容

第5回・第6回の全体像



- 第5回（今日）：左半分。データの成り立ちと変換パイプライン
- 第6回：右半分。3D表示・NDWI等とのオーバーレイ・現地調査ツール化

これまでのタイル配信（第1～2回）の知識がそのまま活きる：

- Terrain RGBタイルの実体は、第1回で扱った地図タイルと同じ
「ズームごとに世界を格子に割って {z}/{x}/{y}.png を並べたもの」
- 違いはPNGの中身が「地図の絵」ではなく「標高値をRGBに詰めた数値」である点だけ
- したがってPMTiles化もNginx静的配信もそのまま使える

Part 1

高さデータはどう作られるか

- 「標高」と一口に言っても、何を測っているかで意味が変わる
- DSM / DTM / CSM の区別
- 3つの主要な計測手法：InSAR・LiDAR・SfM

DSM・DTM・CSMの区別

用語	何の高さか	主な用途
DSM (Digital Surface Model)	建物・樹木を含む地表面	都市解析、電波伝搬、日照
DTM (Digital Terrain Model)	地盤面（構造物・植生を除去）	地形解析、洪水シミュレーション
CSM (Canopy Surface Model)	$DSM - DTM =$ 植生・構造物の高さ	森林資源量、樹高推定

- 日本では「DEM (Digital Elevation Model)」がDTM相当の意味で使われることが多い（基盤地図情報の「数値標高モデル」など）
- 配布データが DSM か DTM かを確認せずに使うと解析結果が大きくずれる**

例：溜池の湛水面積推定にDSMを使うと、堤体周辺の樹木がダムのように振る舞う

計測手法①：LiDAR（レーザー測距）

- 航空機・UAV・地上からレーザーパルスを照射し、往復時間から距離を計測
- マルチリターン：1パルスが樹冠・枝・地面で複数回反射
 - ファーストリターンからDSM、ラストリターン（+地面分類）からDTMを生成
- 出力は点群（LAS/LAZ）→ グリッド化してDEMに

レーザーパルス → 樹冠 (1st return)
 └→ 地面 (last return)

- 国内では林野庁・県による航空レーザー測量成果が公開されつつある
（例：静岡県 VIRTUAL SHIZUOKA、各県のG空間情報センター公開データ）

第4回で扱ったモバイルLiDAR（iPhone等）も原理は同じ。測位精度と到達距離が異なる

計測手法②：SfM / MVS（写真測量）

- Structure from Motion：多視点の写真から特徴点マッチングでカメラ位置と3D形状を同時復元
- UAV空撮 + SfM が現在の低コストDSM生成の主流
- 出力：点群 → メッシュ → オルソ画像・DSM

項目	LiDAR	SfM
植生下の地面	ラストリターンで取得可	取得不可（見えない面は復元できない）
機材コスト	高い	カメラのみで安価
テクスチャ	反射強度のみ	フルカラー
水面・均質面	取得可	特徴点が取れず苦手

SfM由来の「DEM」は実質DSMであることが多い。メタデータでの確認が必要

計測手法③：InSAR（干渉合成開口レーダー）

- SAR衛星（ALOS-2、Sentinel-1等）の位相差から地表の高さ・変位を求める
- 2回の観測の位相差 → 干渉縞（インターフェログラム） → 位相アンラッピング → 高さ
- 世界規模のDEM（SRTM、TanDEM-X由来のAW3D30比較対象等）はこの系譜

特徴：

- 雲を透過する（光学と違い天候に左右されない）
- 広域を一括取得できる一方、解像度は30m級が主流（商用で数m級）
- **差分InSAR (DInSAR)** では地盤沈下・火山性変動をmm～cm精度で検出
→ 高さの「生成」だけでなく「変化の監視」にも使われる

本セミナーの主題である衛星リモートセンシングとの接点が最も深い手法

手法の使い分けまとめ

手法	プラットフォーム	解像度感	得意	不得意
InSAR	衛星	30m～数m	広域・全天候・変位監視	局所の高解像度
LiDAR	航空機・UAV	数十cm～1m	植生下のDTM、高精度	コスト、取得範囲
SfM	UAV・地上	数cm～	低コスト、色情報	植生下、水面

実務では組み合わせる：
広域はSRTM/AW3D30（InSAR/光学立体視由来）、対象地区は県公開の航空LiDAR、
現地の詳細はUAV-SfM、という階層構成が典型

高精細DEMが変えた地形判読：CS立体図・赤色立体図

航空LiDAR由来の高解像度DTMが普及して初めて成立した可視化表現：

表現	考案	作り方の骨子	得意分野
赤色立体図	アジア航測（千葉達朗氏）	傾斜を赤の彩度、尾根谷度を明度に割当	火山地形、微地形の一枚判読
CS立体図	長野県林業総合センター	曲率(Curvature)+傾斜(Slope)を標高段彩に重畳	林内の路網設計、崩壊地・地すべり判読

- 陰影起伏（hillshade）と違い**光源方向に依存しない** → 谷の向きで見落としが出ない
- 樹木を除去したDTMに適用することで、**植生下の微地形**（治山堰堤・旧河道・城郭遺構）が机上で判読できるようになった
- 林業・砂防分野で判読作業の標準になりつつあり、「Q地図タイル」等で全国のCS立体図タイルが公開されている
→ 自作せずタイルとして重ねる使い方も可能

赤色立体図は特許化されている表現（利用条件の確認が必要）。CS立体図は手法が公開されておりGDAL/QGISで自作できる。第6回のオーバーレイ素材としても扱う

Part 2

フォーマットの構造

- GeoTIFF：ラスターDEMの標準
- COG：クラウド・Web時代のGeoTIFF
- LAS / LAZ：点群の標準（第4回の復習を兼ねて）

GeoTIFFの構造

TIFF（タグ付き画像形式）に**地理参照タグ**を追加したもの

GeoTIFF

```
├─ IFD（タグの目録）
│   └─ 画像サイズ・バンド数・データ型（Int16 / Float32 ...）
│   └─ ModelTiepointTag      ← 画像座標と地理座標の対応点
│   └─ ModelPixelScaleTag    ← 1ピクセルの地上サイズ
│   └─ GeoKeyDirectoryTag    ← CRS（EPSGコード等）
└─ 画像データ本体（ストリップ or タイル配置）
```

- DEMでは1バンド・Float32（標高値そのまま）や Int16 が典型
- NoData値（-9999等）の扱いが後段の変換で重要になる
- 第1回で扱った「タイル」とは別に、ファイル内部にもタイル/ストリップ構造がある

gdalinfoで構造を読む

```
gdalinfo dem.tif
```

確認すべき項目：

項目	見る場所	なぜ重要か
CRS	Coordinate System is:	変換の起点。JGD2011平面直角座標系か地理座標か
解像度	Pixel Size	リサンプリングの要否判断
データ型	Type=Float32 等	Terrain RGB変換時のエンコード精度
NoData	NoData Value=-9999	海域・欠測の透過処理
統計値	-stats オプション	標高レンジの妥当性確認（外れ値検出）

外部サービスに投げる前に、まず gdalinfo で自分の目でヘッダを読む習慣をつける

COG (Cloud Optimized GeoTIFF)

通常のGeoTIFFと100%互換でありながら、HTTP Range Requestで部分読み出しできるよう内部配置を規約化したもの

COG の内部レイアウト

- └─ ヘッダ・IFD群（ファイル先頭に集約）
- └─ オーバービュー（縮小版ピラミッド） ← 低ズームはここだけ読む
- └─ フルレゾリューションデータ（512x512等の内部タイル）

- **第2回のPMTiles静的配信と同じ発想**：サーバーは静的ファイルを置くだけ、クライアントがRangeで必要な範囲だけ取得
- Nginxに置くだけで配信でき、動的タイルサーバーが不要になるケースがある
- 生成： `gdal_translate -of COG` または `rio cogeotiff create`

「PMTilesはタイルの詰め合わせをRangeで読む」「COGはラスター本体をRangeで読む」という対比で整理できる

LAS / LAZ（点群フォーマット・第4回の復習）

形式	内容
LAS	点群の非圧縮標準。ヘッダ + 点レコード（XYZ・強度・リターン番号・分類コード）
LAZ	LASの可逆圧縮版。1/7～1/10程度に縮む。laszip / PDAL で相互変換
COPC	LAZ内部を八分木配置しRange Request対応にしたもの（COGの点群版）

中身の確認（バイナリだがヘッダはツールで読める）：

```
pdal info --metadata input.laz
```

```
{ "count": 12873456, "srs": {"horizontal": "EPSG:6669", ...},
  "minz": 2.31, "maxz": 187.42,
  "creation_year": 2023, "point_format": 6 }
```

- 点レコードの**分類コード**（2=地面, 5=高植生 等）でフィルタ → 地面点のみでDTM
- `pdal` の `writers.gdal` や `gdal_grid` でグリッド化 → GeoTIFF化

今日のハンズオンは配布GeoTIFFから始めるため点群処理は扱わないが、上流はこうなっている

フォーマットと配信方式の対応関係

データ	ファイル形式	Web配信形態	対応する回
ベクター	GeoJSON	MVT / PMTiles	第1～2回
点群	LAS / LAZ	Potree / COPC / 3D Tiles	第4回
ラスター(画像)	GeoTIFF	XYZラスタータイル / COG	第1～2回
ラスター(標高)	GeoTIFF (DEM)	Terrain RGBタイル / COG	第5回（今日）

高さデータの配信は「これまでの仕組みの応用」であり、新しいのは
標高値をどうやってPNGに詰めるか（エンコード）の一点だけ

Terrain RGBエンコード

PNGタイルは8bit×3チャンネル（RGB）しか持てない
→ 24bitに標高値をエンコードして詰め込む

Mapbox Terrain-RGB方式：

```
height = -10000 + ((R * 256 * 256 + G * 256 + B) * 0.1)
```

- 分解能 0.1m、範囲 -10,000m～+約1,667,721m
- MapLibre / deck.gl がデコードを標準搭載

****地理院標高タイル（PNG形式）**は別方式（分解能0.01m、無効値表現も異なる）**

両者に互換性はない。MapLibreのterrainに地理院標高PNGをそのまま渡すと壊れた地形になる

encoding: "gsi" 相当の変換・カスタムデコードの話は第6回で扱う

Part 3

入手できる高さデータソース

- グローバル：SRTM / AW3D30
- 国内：基盤地図情報・県の航空LiDAR成果
- ライセンスと解像度の確認ポイント

主要な高さデータソース

データ	提供	解像度	入手形式（拡張子）	由来・備考
SRTM	NASA/USGS	30m (1秒)	.hgt / GeoTIFF	シャトルInSAR (2000年)。取得年が古い
AW3D30	JAXA	30m	GeoTIFF (.tif)	ALOS光学立体視。全球・無償版
基盤地図情報 数値標高モデル	国土地理院	10m / 5m / 1m	JPGIS GML (.xml)	5m/1mは整備範囲に差
地理院 標高タイル	国土地理院	ズーム別	.txt / .png タイル	PNG方式は独自エンコード
GSI 1mメッシュDEM	国土地理院	1m	公開形態を要確認	整備範囲拡大中（後述）
県公開LiDAR点群	各県	点密度依存	.las / .laz	G空間情報センター等

確認の手順：**解像度** → **DSM/DTMの別** → **取得年** → **CRS・標高基準** → **形式** → **ライセンス**

ここで一度手を止めて、実際にG空間情報センター・基盤地図情報DLサイトを一緒にブラウジングします（5分）

中身を覗く：テキストで読める高さデータもある

基盤地図情報 数値標高モデル (JPGIS GML / .xml)：グリッド値がテキストで並ぶ

```
<gml:tupleList>
gnd,135.42
gnd,135.51
gnd,135.63    ← 「種別, 標高値」が西→東、北→南の順に1点1行
...
</gml:tupleList>
```

地理院標高タイル (テキスト形式 / .txt)：256×256のCSVそのもの

```
168.3,168.1,167.8,...    ← 1行が256列。値は標高[m]、無効値は "e"
169.0,168.7,e,e,...
```

- テキストで読める形式は中身の意味を目で確かめる教材として最適
- 一方で容量・パース速度の面でWeb配信には不向き → だからバイナリ形式とエンコードが要る

形式ごとに「Webでどう使うか」の対応表

入手形式	そのままWebで使えるか	Webで使うための経路
JPGIS GML (.xml)	使えない	GeoTIFFに変換（QGIS / 変換ツール）→ 本日のパイプラインへ
GeoTIFF (.tif)	ほぼ不可（巨大・非タイル）	COG化してRange配信 or Terrain RGBタイル化
COG	使える	<code>gdal_translate -of COG</code> 。クライアント側でRange読み
地理院標高タイルPNG	MapLibre標準では不可	独自エンコードのためカスタムデコードが必要
Terrain RGB / Terrarium タイル	使える	MapLibre <code>raster-dem</code> が標準対応。本日の到達点
LAS/LAZ 点群	使えない	PDALでDEM化 → 上記へ合流 / 点群のままなら第4回の経路

「入手形式」と「Web配信形式」は別物。この間を埋めるのがGDALパイプライン

公開タイルをそのまま使う選択肢：Mapterhorn

自前変換の対極として、**全球の地形タイルセットを静的PMTilesで公開**するプロジェクトが登場している（2025年～、Protomaps周辺コミュニティ）

- [Mapterhorn](#)：Copernicus DEM・swissALTI3D等のオープンDEMを統合しタイル化
- **Terrariumエンコード・512pxタイル・WebP** で配信。MapLibreが標準デコード対応

```
terrainSource: {  
  type: "raster-dem",  
  tiles: ["https://tiles.mapterhorn.com/{z}/{x}/{y}.webp"],  
  encoding: "terrarium",    // Mapbox方式ではない点に注意  
  tileSize: 512             // 256のままだと地形が破綻する典型ハマりポイント  
}
```

- `pmtiles extract --bbox=...` で**必要範囲だけ切り出して自前ホスト**もできる
→ 第2回の静的配信構成にそのまま載る。本日のパイプラインの「完成形の見本」

ジオイド問題への現代的な回答：Re:earth Terrain

Part 3後半で触れる「楕円体高と標高のずれ」に正面から取り組んだ国産プロジェクト

- terrain.reearth.land (Re:earth / Eukarya)
- Mapterhorn由来のDEMに **EGM2008ジオイド**をリクエスト時に合成し、レンダラーが描いている楕円体面に高さを合わせて配信する
- 出力形式は3種：**quantized-mesh-1.0** (Cesium向け) / **Mapbox Terrain-RGB** / **Terrarium**

使い分けの目安：

- 通常の2.5D地形表示 (MapLibre) → Mapterhornで十分
- **GNSS実測値との比較・Cesiumシーンとの合成**など楕円体高が要る場面 → Re:earth Terrain

「同じDEMでも、どの基準面の高さとして配るか」という設計の違いが形式選択に現れる好例

ライブラリ別：DEMをどう食わせるか（第6回への見取り図）

ライブラリ	DEMの入口	一言メモ
MapLibre	raster-dem ソース + setTerrain / hillshade	Terrain RGB / Terrariumを標準デコード。最短経路
deck.gl	TerrainLayer (elevationData + texture)	エンコード式を elevationDecoder で自分で指定する設計
CesiumJS	quantized-mesh (Terrain Provider)	ラスタータイルではなくメッシュ。Re:earth Terrainが直結
Three.js	自前：タイルPNG取得→デコード→PlaneGeometry 頂点変位	1メッシュ分だけ高精度DEMを読むような小回りが利く
AR.js	DEM値をカメラ座標系の高さに変換して重畳	方位ずれ補正が必須（第3回の補正コントローラ実装を参照）

- Three.js経路は「タイルの中身が数値である」ことを最も実感できる教材
- AR適用は第6回で扱う予定（第3回実装の続き）
- 実装例の宝庫：github.com/shiwaku（shi worksさんの一連のデモ。MapLibre/deck.gl/Cesium/3DGSまで網羅的）

全域公開の点群から「自分でDSMを作る」

近年、都県単位で航空LiDAR点群そのものがオープンデータ化されている：

提供	名称・公開先	形式
静岡県	VIRTUAL SHIZUOKA（G空間情報センター）	LAS/LAZ
東京都	都デジタルツイン実現プロジェクトの点群データ	LAS 等
長崎県	オープンナガサキ（G空間情報センター）	LAS 等

意味するところ：DEM/DSMを「配布された完成品」として受け取るのではなく、
点群から自分の目的に合った高さモデルを切り出して作れる時代になった

- 分類コードの選び方次第で DSM にも DTM にも CSM にもなる（Part 1の復習）
- 取得年・点密度を自分で確認でき、更新差分（伐採・造成の検出）にも使える

点群 → DSM のグリッド化パイプライン

```
# 対象範囲のLAZをダウンロード後、PDALでグリッド化  
pdal pipeline dsm.json
```

```
{ "pipeline": [  
  "input.laz",  
  { "type": "filters.range", "limits": "returnnumber[1:1]" },  
  { "type": "writers.gdal",  
    "filename": "dsm.tif",  
    "resolution": 1.0,  
    "output_type": "max",  
    "nodata": -9999 }  
]}
```

- ファーストリターン + `max` → **DSM** / `Classification[2:2]` + `idw` → **DTM**
- 出力はGeoTIFFなので、**ここから先は今日のGDALパイプラインにそのまま合流する**

時間の都合で本日は流れの紹介まで。PDALパイプラインの詳細は `handson/08_pointcloud_dsm` を参照（第4回の点群処理の続きとしても読める）

標高の「基準」にも注意

- 楕円体高：GNSS/衛星が直接測るのはこちら（GRS80楕円体からの高さ）
- 標高（正標高）：ジオイド面（平均海面）からの高さ。地図・行政データはこちら
- 差＝ジオイド高。日本では約30～40m

$$\text{楕円体高} = \text{標高} + \text{ジオイド高}$$

- UAV-SfMの成果（GNSS基準）と基盤地図情報（標高）を混ぜると数十mずれる
- 変換には国土地理院のジオイドモデル（GSIGEO2024 等）を使う

「高さが30mずれている」場合、まずジオイド高の混入を疑う

Part 4 ハンズオン

GDAL変換パイプラインの実装

配布GeoTIFF（DEM） → 前処理 → Terrain RGB → タイル書き出し

```
dem.tif → [gdalwarp: 座標変換+クリップ+リサンプリング]  
        → [gdal_translate / rio: COG化]  
        → [rio rgbify: Terrain RGBエンコード]  
        → [タイル書き出し → PMTiles化]
```

資料： 2026-07-24/hands-on/ 各ディレクトリのreadme.mdに沿って進める

環境準備（00_setup）

```
# GDALの確認（3.4以降を想定）  
gdalinfo --version  
  
# Python系ツール（Terrain RGB変換に使用）  
pip install rio-cogeo rio-rgbify  
  
# PMTiles変換（第1回で導入済みの環境を流用）  
pmtiles version
```

- GDALはOSGeo4W（Windows） / Homebrew（macOS） / apt（Linux）で導入
- QGIS同梱のGDALをコマンドラインから使う方法も資料に記載
- **rio-rgbify はメンテナンス頻度が低い。動作しない場合の代替（gdal_calc による自前エンコード）も 04 の資料に用意**

Step 1：データの確認と座標変換（gdalwarp）

```
# まず中身を確認する（Part 2 の復習）
gdalinfo -stats dem_src.tif

# JGD2011 地理座標 → WebMercator(EPSG:3857) へ変換しつつ範囲を切り出す
gdalwarp \
  -s_srs EPSG:6668 -t_srs EPSG:3857 \
  -te 14640000 3980000 14680000 4020000 -te_srs EPSG:3857 \
  -r bilinear \
  -dstnodata -9999 \
  dem_src.tif dem_3857.tif
```

- `-r bilinear`：標高など連続値は**bilinear/cubic**。**nearest**はカテゴリ値（土地利用等）専用
- `-te` で処理範囲を絞ってから後段に進む（フル範囲での試行錯誤は時間の無駄）

Step 2：リサンプリングとCOG化

```
# 解像度を10mに揃える（過剰な解像度はタイルサイズを無駄に増やす）
gdalwarp -tr 10 10 -r bilinear dem_3857.tif dem_10m.tif

# COGとして書き出し（オーバービュー自動生成）
rio cogeo create dem_10m.tif dem_cog.tif

# 検証
rio cogeo validate dem_cog.tif
```

- COGはこの時点でNginxに置けば配信可能（第6回でCOG直読みも試す）
- `gdaladdo` で明示的にオーバービューを作る方法も資料に記載

Step 3 : Terrain RGBエンコードとタイル化

```
# 標高値 → Mapbox Terrain-RGB 方式のRGBへエンコードしつつmbtiles化
rio rgbify \
  --min-z 8 --max-z 14 \
  --interval 0.1 \
  --format png \
  dem_cog.tif terrain.mbtiles

# mbtiles → PMTiles（第2回の静的配信構成にそのまま載る）
pmtiles convert terrain.mbtiles terrain.pmtiles
```

- `--interval 0.1` : エンコード分解能 (0.1m)。式の `* 0.1` に対応
- `max-z` はデータ解像度に見合う値に : 10m DEMにz18は無意味 (1pxが実解像度を超える)
- **NoData領域の縁がノイズ状の崖になる場合がある** → 資料でマスク処理を解説

Step 4：動作確認

```
# 第2回で構築したNginx構成に terrain.pmtiles を配置  
# ローカル確認は python -m http.server でも可
```

確認用の最小ページ（`06_preview/index.html`）で以下を確認：

- MapLibreの `raster-dem` ソースとしてPMTilesを読み込み
- `hillshade` レイヤーで陰影段彩表示 → **標高値が正しくデコードされているかの目視検証**
- おかしな縞・崖が出る場合：エンコード方式の不一致 or NoData処理漏れを疑う

terrain（3D表示）まで踏み込むのは第6回。今日は「正しいタイルが作れた」ことの検証まで

発展：GSI 1mメッシュDEMで山口県内を高精細化

国土地理院が1mメッシュDEMの整備・公開を進めている（整備範囲は順次拡大中）

- 公開情報：gsi.go.jp/gazochosa/gazochosa61005.html
- 本講座では宇部新川駅周辺・あすとぴあ周辺を対象範囲として用意

10mとの違いが最も出るのは：

- 河川堤防・道路の切土盛土・ため池の堤体といった人工地形の輪郭
- max-z の設計：1m解像度なら **z16～17まで意味がある**（10mではz14が上限の目安）

手順は本編と同一。max-z と対象範囲だけが変わる

```
rio rgbify --min-z 10 --max-z 16 --interval 0.1 dem1m_cog.tif terrain1m.mbtiles
```

処理時間・容量が10m比で大きく増える。ハンズオン中は完走できなくてよい

（`handson/07_gsi1m_yamaguchi` に手順一式。自宅で完走することを想定した構成）

パイプライン全体の再掲

```
dem_src.tif
| gdalinfo -stats          ← 構造・CRS・NoDataの確認
| gdalwarp                 ← EPSG:3857化・クリップ・bilinear
▼
dem_10m.tif
| rio cogeo create         ← COG化（この時点で配信可能）
▼
dem_cog.tif
| rio rgbify               ← 標高値→RGB 24bitエンコード
▼
terrain.mbtiles
| pmtiles convert          ← 第2回の配信構成へ
▼
terrain.pmtiles → Nginx静的配信 → MapLibre hillshadeで検証
```

各段の中間ファイルを残しておくと、問題の切り分けが段単位でできる

よくあるつまずきどころ

症状	原因の候補	確認方法
地形が階段状	nearestでリサンプリングした	gdalwarpの <code>-r</code> を確認
全体が海面下/異常な高さ	エンコード方式の不一致（地理院PNG等）	デコード式と <code>--interval</code> を照合
タイル境界に筋	NoData縁の処理漏れ	<code>-dstnodata</code> とマスクを確認
高さが約30～40mずれる	楕円体高とジオイド高の混同	元データの標高基準を確認
ファイルが巨大	max-zが解像度に対し過剰	1pxの地上サイズと元解像度を比較

本日のまとめ

- 高さデータは **何を（DSM/DTM）・どう測ったか（InSAR/LiDAR/SfM）** で性質が決まる
- GeoTIFF/COG/LAS/LAZ の内部構造は gdalinfo 等で自分で読める
- COGとPMTilesは「静的ファイル + Range Request」という同じ設計思想
- Terrain RGBは標高値を24bit RGBに詰めるエンコードであり、方式の互換性に注意
- GDALパイプラインの各段で中間成果を検証する習慣が、問題の切り分けを容易にする

次回予告

第6回 高さデータ②：可視化・実務データとのオーバーラップ・現地調査への応用

- 今日作った `terrain.pmtiles` を MapLibre terrain / deck.gl TerrainLayer で3D表示
- NDWIラスター・防災重点溜池（国土数値情報）とのオーバーレイ
- 垂直誇張・hillshadeのパラメータ設計
- スマホブラウザからそのまま現地で使える形に仕上げる